Análisis y arquitectura AWS de coste mínimo para AELB

1. Lo que hay realmente en el repositorio

Frontend — `aelb-frontend/`: Next.js 16 + React 19 + TypeScript + Tailwind 4, `next-intl` (es/en). Hoy se ejecuta como **servidor Node** (`output: 'standalone'`, `node server.js`) porque `/campeonatos/[id]` es una ruta dinámica cuyo ID no existe en build-time. Vuestro propio `docs/arquitectura-y-despliegue.md` ya documenta esto y propone la salida a `output: 'export'` (estático) cambiando a `/campeonato?id={id}` — es la vía correcta para poder servir el front desde S3+CloudFront, pero hoy no está hecha.

Backend — `backendweb/`: Java 21, Spring Boot 4.0.6, Maven, arquitectura hexagonal (`domain/application/infrastructure`). Spring Security + JWT propio (access 15 min + refresh rotatorio 30 días en tabla `refresh_tokens`), Actuator, AWS SDK v2 (`s3`) ya integrado. **No hay `@Scheduled`, `@Async`, WebSockets ni envío de email en ningún sitio del código.** El único proceso "batch" es `LegacyFileMigrationRunner`, un runner de un solo uso activado por el perfil `storage-migration` para migrar ficheros legacy a S3 — no es un job recurrente.

Base de datos: MySQL 8.4, 17 tablas (`schema.sql`, 412 líneas), 12 migraciones manuales. `hibernate.ddl-auto=validate` — el esquema se gestiona con SQL a mano, no con Flyway/Liquibase. Conexión TLS `VERIFY_IDENTITY` en producción.

Almacenamiento de ficheros: ya está resuelto correctamente. `infrastructure/storage/` (S3FileStorage, PresignedUrlService, StorageInitializer, StoredFileKeys) sube imágenes/PDFs a S3/MinIO, MySQL solo guarda `*_storage_key`. Soporta **`STORAGE_USE_IAM=true`** para usar rol IAM de instancia en vez de claves estáticas — esto es relevante para la arquitectura: el código ya está listo para EC2 con instance profile, sin credenciales en el entorno.

Pagos: Stripe Checkout (cuotas anuales + inscripciones), con webhook `POST /api/webhooks/stripe` que debe recibir el *raw body* sin transformar, ser público y no llevar JWT. Circuit breaker y reintentos ya implementados. Requiere: dominio HTTPS público, y salida HTTPS saliente a `api.stripe.com`.

Sin encontrar: emails, colas, WebSockets, tareas programadas, CI/CD (`.github/` no existe), IaC (no hay Terraform/CloudFormation). Es un proyecto sin infraestructura AWS todavía — partimos de cero, sin nada que migrar.

Detalle arquitectónico importante para el dimensionamiento: `InMemoryRateLimiter` guarda estado en memoria del proceso. Esto significa que **hoy el backend solo puede correr como una única instancia** — escalar horizontalmente a 2+ réplicas rompería el rate limiting sin antes moverlo a Redis/DB. Encaja perfectamente con la prioridad de coste: una sola instancia pequeña es tanto la opción más barata como la única correcta con el código actual.

2. Recursos reales necesarios

Con 5–10 usuarios concurrentes y picos de 80–150 inscripciones puntuales:

Recurso	Estimación

CPU	Prácticamente en reposo el 95% del tiempo; picos cortos y ligeros durante aperturas de inscripción. Una CPU compartida (burstable, tipo `t4g`) sobra.
RAM	JVM Spring Boot ajustada (`-Xmx256m–384m`) + Next.js Node (~~150-200 MB) + MySQL (~~`innodb_buffer_pool_size`~~128-256 MB) + proxy (~~20-30 MB) ≈ 600–750 MB en uso real.
Conexiones DB	Pool de 5-10 conexiones es más que suficiente; nunca se acercará a los límites por defecto de MySQL.
Almacenamiento BD	Esquema de 17 tablas, cientos de usuarios/inscripciones al año → varios cientos de MB tras varios años. Insignificante.
Ficheros S3	Logos, PDFs de resultados, documentos por campeonato: probablemente unos pocos GB acumulados en años. Coste S3 irrelevante (menos de 1 €/mes).
Transferencia	Tráfico de una app de gestión con 5-10 usuarios simultáneos: unos pocos GB/mes salientes. Muy por debajo de cualquier umbral de coste real.
Requests	Cientos/día en uso normal, quizá unos miles en un pico de apertura de inscripciones. Trivial para cualquier instancia pequeña.

Conclusión: esto no necesita ni siquiera un `t3.small` tradicional. El único recurso que realmente hay que vigilar es la **RAM combinada** de JVM + Node + MySQL en la misma máquina si se opta por co-alojar todo.

3. Comparación de alternativas

A — EC2 pequeña (Graviton): Backend + proxy + frontend (+ opcionalmente MySQL) en una sola `t4g.micro` / `t4g.small` con Docker Compose. Coste mínimo, control total, encaja con el `docker-compose.yml` que ya existe (solo hay que adaptarlo a producción). Riesgo de co-alojar la BD: aceptable a este tamaño si hay backups automatizados; es la única forma de mantenerse en el rango de coste pedido.

B — Lightsail: Facturación empaquetada (cómputo+disco+IP+transferencia en un precio fijo), más simple de presupuestar que EC2 a la carta. El plan de 2 GB RAM (~12 $/mes) es equivalente en specs a un `t4g.small`. Contras frente a EC2: no soporta roles IAM de instancia de forma nativa (habría que usar claves de acceso IAM estáticas para S3, cuando el código ya está preparado para NO necesitarlas), y los snapshots automáticos y la BD gestionada de Lightsail son extras que encarecen. Buena opción "por si EC2 os intimida"; peor encaje técnico que EC2 dado el código actual.

C — Serverless (S3+CloudFront / API Gateway+Lambda / Aurora Serverless o DynamoDB): Descartado para el backend. Spring Boot en Lambda tiene cold starts de varios segundos (la app no está preparada como handler Lambda, exigiría reescritura con `aws-serverless-java-container` o SnapStart, esfuerzo no trivial), y con 5-10 usuarios activos de forma más o menos continua el ahorro de "pagar solo por invocación" es marginal frente al coste en latencia/complejidad. Aurora Serverless v2 tiene un suelo de coste (mínimo 0.5 ACU) que ronda 30-40 €/mes solo de BD — más caro que EC2 completo. DynamoDB exigiría remodelar 17 tablas relacionales con FKs y UNIQUE compuestos (doble inscripción, cuota única por año) que hoy dependen de constraints de MySQL: reescritura de dominio, no un cambio de infraestructura. **No compensa aquí.**

D — ECS Fargate + ALB + RDS: Coste fijo antes de servir una sola petición: ALB (~~16-18 €/mes) + Fargate mínimo 24/7 (~~9 €/mes por tarea) + RDS (~15-18 €/mes) ≈ **45-55 €/mes solo en infraestructura base**, más del doble del presupuesto objetivo, para una app con picos de 5-10 usuarios. Sobredimensionado.

E — App Runner: Simplifica el despliegue (build desde Dockerfile/ECR sin gestionar servidores), pero el coste de cómputo activo más una RDS aparte sitúan el total en 25-40+ €/mes, y si se usa "scale to zero" para bajar coste, el cold start de Spring Boot penaliza al primer usuario tras inactividad. No aporta suficiente frente a EC2 para este tamaño y presupuesto.

F — Híbrida económica (S3+CloudFront para frontend + EC2/Lightsail pequeña + S3 ficheros): La mejor relación coste/sencillez/seguridad, y es literalmente lo que vuestra propia documentación ya apunta como plan de ahorro futuro. Requiere primero el cambio de `/campeonatos/[id]` a ruta con query param para poder exportar estático — se deja como fase 2 (ver §16). Mientras tanto, Next.js server + EC2 es perfectamente válido y barato.

4. Costes ocultos a evitar

Servicio	Por qué evitarlo aquí
NAT Gateway	~33 €/mes fijos + tráfico. La instancia puede ir en subred pública con Security Group restrictivo; si se quisiera subred privada, un **VPC Gateway Endpoint de S3 es gratuito** y evita el NAT igualmente.
Application Load Balancer	~16-18 €/mes fijos. Con una sola instancia, un proxy en la propia máquina (Caddy/Nginx) hace el mismo trabajo (TLS + routing) a coste cero.
IP pública / Elastic IP	Desde 2024 AWS cobra **~3,65 €/mes por cualquier IPv4 pública**, esté en un EIP o asignada automáticamente a la instancia. Es inevitable si se quiere acceso público directo; ya se incluye en la estimación.
RDS	Gestión y backups automáticos, pero ~15-18 €/mes solo de instancia+storage — se come todo el presupuesto. Se pospone a la fase de escalado (§14).
CloudWatch Logs	Sin agente de logs no hay ingesta que pagar. Usar `docker logs`/journald local con rotación es gratis y suficiente a este volumen.
Secrets Manager	0,40 $/secreto/mes. Con 4-5 secretos son ~2 €/mes evitables: **SSM Parameter Store (tier estándar) es gratis** y cifra con KMS igualmente.
ECR	Coste real insignificante (imágenes de ~150-250 MB, céntimos/mes) y aporta versionado + `docker pull` con el mismo rol IAM que ya se usa para S3, sin credenciales estáticas. Vale la pena, no es un coste oculto real.
Route 53 hosted zone	0,50 €/mes, evitable si el dominio ya está en otro registrador: solo apuntar un registro A/ALIAS a la IP de la instancia.
WAF	Innecesario a este tráfico; el Security Group + rate limiting ya en el código cubre lo razonable.
Snapshots/backups múltiples	Backups sí, pero uno diario con retención corta (7-30 días) en S3 con lifecycle a Glacier o borrado — no múltiples políticas solapadas.

5. Base de datos: MySQL en la propia instancia

Con 17 tablas y este volumen, **MySQL en contenedor Docker en la misma EC2, con volumen EBS y `mysqldump` nocturno a S3**, es la opción correcta ahora:

- Coste: prácticamente cero adicional (unos céntimos de EBS/S3).

- Riesgo asumible: recuperación manual, sin failover automático — pero con 5-10 usuarios y este tráfico, una restauración de minutos ante un incidente es tolerable.
- Mitigación: `mysqldump` diario cifrado a un bucket S3 privado con versioning + lifecycle, y snapshot semanal de EBS vía **AWS Backup/DLM (solo se paga el storage del snapshot, el servicio es gratis)**.

RDS queda como upgrade claro cuando se pase a 50 usuarios concurrentes o cuando se quiera backup/failover gestionado sin operarlo a mano (§14).

6. Frontend

Fase 1 (ya mismo, sin tocar código): mantener Next.js server en Docker en la misma EC2, detrás del proxy. Coste marginal cero adicional (ya cabe en la RAM presupuestada).

Fase 2 (optimización, recomendable pero opcional): aplicar el cambio que la documentación del proyecto ya propone (`/campeonato?id={id}` en vez de `/campeonatos/[id]`), volver a `output: 'export'` , y servir desde **S3 + CloudFront**. Con este tráfico, CloudFront cae dentro o muy cerca del *free tier* (1 TB/mes durante 12 meses, luego ~0,085 $/GB) — coste estimado **0-2 €/mes**. Beneficio adicional: libera ~150-250 MB de RAM en la instancia, permitiendo bajar de `t4g.small` a `t4g.micro` sin apuros.

7. Archivos e imágenes

Ya resuelto en el código actual — solo falta la parte AWS:

```
s3://aelb-files-prod/
  championships/{championshipId}/documents/{uuid}-{filename}
  championships/{championshipId}/results/{uuid}.pdf
  users/{userId}/...
```

Bucket privado, cifrado SSE-S3 por defecto, "Block Public Access" activado, acceso solo vía rol IAM de la instancia (`STORAGE_USE_IAM=true`) y URLs presignadas — exactamente lo que ya describe `docs/arquitectura-y-despliegue.md` . No hace falta CloudFront delante de este bucket a este volumen.

8. Dominio y HTTPS

Recomendación concreta: **un único dominio** (`app.aelb.es`) sirviendo tanto el frontend como `/api/*` a través del mismo proxy en la instancia (Caddy). Esto evita CORS (mismo origen) y encaja con `AUTH_COOKIE_SAME_SITE=Strict` , que hoy exige same-site para que la cookie JWT funcione sin fricciones.

- **DNS**: registro A/ALIAS al IP público de la instancia. Route 53 opcional (0,50 €/mes) si se quiere gestionar desde AWS; si no, el DNS del registrador actual sirve igual y gratis.
- **HTTPS**: **Caddy** como proxy inverso, con renovación automática de Let's Encrypt (gratis, cero mantenimiento). Si más adelante se añade CloudFront para el frontend estático, ahí el certificado es **ACM gratuito**.

9. Seguridad (mínimos, sin servicios enterprise)

- HTTPS obligatorio en todo (Caddy/Let's Encrypt).
- MySQL solo accesible desde `localhost` /red interna Docker, nunca expuesto a Internet (sin puerto 3306 en el Security Group).
- Secretos (`JWT_SECRET` , `STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET` , credenciales DB) en **SSM Parameter Store (SecureString, gratis)**, no en el repo, inyectados como variables de entorno en el deploy.

- IAM: rol de instancia con permisos mínimos — solo `s3:GetObject/PutObject/DeleteObject` sobre el bucket concreto, y `ecr:GetDownloadUrlForLayer` / `BatchGetImage` para hacer `docker pull`. Nada más.
- Backups automáticos (§5).
- Security Group: solo 80/443 públicos. **Sin puerto 22 abierto** — administración vía **AWS Systems Manager Session Manager** (gratis, sin credenciales SSH que gestionar, sin superficie de ataque en el puerto 22).
- El webhook de Stripe (`/api/webhooks/stripe`) debe quedar excluido del filtro JWT (ya lo está, según la documentación) y pasar el *raw body* sin que el proxy lo transforme — Caddy no toca el body por defecto, así que es compatible.
- Logs mínimos: `docker logs` /journald con rotación local (`max-size` , `max-file`), sin enviar a CloudWatch salvo que se necesite depurar algo puntual.

10. CI/CD

```
GitHub Actions (push a main)
  -> mvn package (backend) / npm run build (frontend)
  -> docker build (ambas imágenes, ya existen Dockerfiles multi-stage)
  -> docker push a ECR (rol IAM vía OIDC, sin claves AWS guardadas en GitHub)
  -> aws ssm send-command -> en la instancia: docker compose pull && docker compose up
-d
```

Se usa ECR en vez de copiar el artefacto/tarball a mano porque el coste es insignificante (céntimos/mes) y reutiliza el mismo modelo de "sin credenciales estáticas" que ya se usa para S3 — tanto el pull de imagen como el acceso a S3 usan el rol IAM de la instancia. `aws ssm send-command` sustituye a SSH: no hay claves privadas en secretos de GitHub ni puerto 22 abierto.

Las migraciones SQL (dado que `ddl-auto=validate`) se siguen aplicando explícitamente, no automáticamente: un pequeño script `deploy/migrate.sh` que aplica los ficheros de `database/migrations/` pendientes contra la BD antes de levantar el nuevo contenedor backend, igual que se hace manualmente hoy.

11. Entornos

Nada de DEV/PRE permanentes en AWS. Propuesta:

- **LOCAL**: `docker-compose.yml` actual (MySQL + MinIO + backend + frontend) — ya cubre el 90% de las validaciones, incluido Stripe en modo test con `stripe listen` (documentado en `docs/stripe-produccion.md`).
- **PRO**: la única instancia en AWS descrita más abajo.
- Si alguna vez hace falta un entorno de verificación real (p. ej. antes de activar Stripe live), levantar una **segunda EC2/Lightsail temporal** unas horas o días y terminarla después — coste de céntimos, no una infraestructura paralela permanente.

12. Estimación económica (región eu-west-1, Irlanda)

Precios verificados a fecha actual (agosto 2026); USD y EUR se tratan aproximadamente a la par en esta tabla.

Arquitectura	Coste estimado/mes	Complejidad	Escalabilidad

Lightsail (2 GB, todo incluido)	12-15 €	Baja	Media
EC2 t4g.micro/small + MySQL + S3 (recomendada)	**13-19 €**	Media	Media
S3 + CloudFront + EC2 (fase 2, front estático)	10-15 €	Media	Alta
Serverless (Lambda+API GW+Aurora Serverless)	45-60 €+ (y cold starts)	Media/Alta	Muy alta
ECS Fargate + ALB + RDS	55-75 €	Alta	Muy alta

Desglose de la opción recomendada (on-demand, sin compromiso):

Componente	Coste/mes
EC2 t4g.small (2 vCPU, 2 GB RAM, ARM)	~11,5 €
IPv4 pública	~3,4 €
EBS gp3 20 GB	~1,5 €
S3 (ficheros + backups)	~0,5-1 €
Transferencia saliente	~0,5 €
Total	**~17-18 €/mes**

Con `t4g.micro` (1 GB, requiere afinar JVM y activar swap) o con un compromiso de 1 año (Savings Plan, ~~30% descuento en cómputo):~~ **10-13 €/mes**.

Nota de región: `eu-south-2` (España) suele tener un sobrecoste del 10-20% frente a `eu-west-1` / `eu-central-1` por ser una región más nueva y con menor escala. Para una app de gestión sin requisitos de latencia crítica (no es tiempo real), se recomienda **`eu-west-1` (Irlanda)** por precio, o **`eu-west-3` (París)** como término medio si se prefiere menor latencia desde España (~15-20 ms) a cambio de un precio muy similar a Irlanda. Se evitaría `eu-south-2` salvo que haya un requisito explícito de residencia de datos en España.

13. Objetivo 5-20 €/mes: ¿viable?

Sí, con margen. La opción recomendada cae en **13-19 €/mes** en el peor caso (on-demand, `t4g.small`), y puede bajar a **~10 €/mes** con `t4g.micro` bien afinada o compromiso de 1 año. El único componente que no se puede eliminar del todo es la **IP pública (~3,65 €/mes obligatorios desde 2024)** y el cómputo mínimo de la instancia; todo lo demás (S3, backups, DNS, Parameter Store, ECR) es prácticamente gratis a este volumen.

14. Escalabilidad futura

- **5-10 -> 50 usuarios concurrentes**: subir de `t4g.small` a `t4g.medium` / `large` (cambio de tipo de instancia, sin tocar arquitectura), y migrar MySQL a **RDS** **`db.t4g.micro/small`** para tener backups/failover gestionados (~+15-20 €/mes). Aplicar entonces el frontend estático en S3+CloudFront (fase 2) para descargar la instancia. El rate limiter en memoria seguiría siendo válido con una sola instancia backend.
- **50 -> 500 usuarios concurrentes**: aquí sí compensa **ECS Fargate (o EC2 Auto Scaling) + ALB + RDS Multi-AZ**, y habría que mover el rate limiter a Redis/ElastiCache para poder escalar horizontalmente

el backend. Es el punto donde el coste fijo de ALB dejaría de ser desproporcionado, porque ya hay tráfico real que lo justifica.

No se paga hoy nada de esa capa intermedia: el salto de EC2 única -> RDS -> Fargate/ALB es incremental y cada paso se activa solo cuando el uso real lo pide.

15. Recomendación final concreta

```
Internet
   |
   +-- DNS (registrador actual o Route 53) -> A record -> IP publica EC2
   |
   HTTPS (Let's Encrypt via Caddy)
   |
EC2 t4g.small (ARM64, Ubuntu 24.04 LTS) - eu-west-1
   |
   +-- Caddy (reverse proxy, TLS)
   |     +-- /            -> Next.js (Docker, standalone server, puerto 3000)
   |     +-- /api/*       -> Spring Boot (Docker, puerto 8080)
   |
   +-- MySQL 8.4 (Docker, volumen EBS gp3 20 GB)
   |
   +-- Rol IAM de instancia (sin claves estaticas)
             +-- S3 (ficheros/documentos, bucket privado)
             +-- S3 (backups mysqldump diarios)
             +-- ECR (pull de imagenes en el deploy)
```

- **Servicio**: EC2, no Lightsail — porque el código ya está preparado para rol IAM de instancia (`STORAGE_USE_IAM=true`) y así no hace falta ninguna credencial AWS estática en el servidor.
- **Tamaño**: `t4g.small` (2 vCPU, 2 GB RAM) por margen de seguridad con JVM+Node+MySQL simultáneos; `t4g.micro` es viable si se afina memoria y se añade swap, pero con menos margen operativo.
- **SO/Arquitectura**: Ubuntu 24.04 LTS, ARM64/Graviton (más barato que x86 para el mismo rendimiento; los Dockerfiles actuales ya usan imágenes base multi-arch).
- **Frontend**: contenedor Next.js standalone en la misma instancia (fase 1); migrar a S3+CloudFront cuando se haga el cambio de ruta dinámica (fase 2, opcional).
- **Backend**: contenedor Spring Boot en la misma instancia.
- **Base de datos**: MySQL en contenedor en la misma instancia, volumen EBS dedicado, backup diario a S3.
- **Ficheros**: S3 privado, acceso vía rol IAM + URLs presignadas (ya implementado).
- **DNS**: registro A al IP público; Route 53 opcional.
- **HTTPS**: Caddy + Let's Encrypt, un solo dominio para front y API (mismo origen, sin CORS).
- **Backups**: `mysqldump` diario -> S3 (lifecycle 30-90 días) + snapshot EBS semanal (DLM).
- **Logs**: `docker logs` /journald con rotación local; sin CloudWatch Logs salvo depuración puntual.
- **Secretos**: SSM Parameter Store (SecureString, gratis), no Secrets Manager.
- **Docker**: Docker Compose en la instancia (reutilizando el `docker-compose.yml` actual con un override de producción), imágenes construidas en CI y publicadas en ECR.
- **CI/CD**: GitHub Actions -> build+test -> push a ECR -> `aws ssm send-command` para `docker compose pull && up -d` en la instancia, sin SSH ni claves AWS de larga duración (OIDC).

16. Plan de implementación

1. **Cambios de código**: ninguno bloqueante para desplegar tal cual está hoy. Opcional (fase 2): refactor de `/campeonatos/[id]` a `/campeonato?id=` para habilitar export estático.
2. **Configuración**: crear `docker-compose.prod.yml` (sin puertos de MinIO/MySQL expuestos al host, `SPRING_PROFILES_ACTIVE` de producción, sin el perfil `local`), y variables leídas desde SSM en vez de `.env` plano.
3. **Dockerización**: ya está hecha (Dockerfiles backend/frontend); solo falta el `Dockerfile` /config de Caddy para el proxy.
4. **Infraestructura AWS**: crear VPC (o usar la default), Security Group (80/443 públicos, resto cerrado), EC2 `t4g.small` con rol IAM (S3 + ECR), bucket S3 (ficheros) y otro/prefijo para backups, repositorios ECR.
5. **DNS**: registro A al IP público de la instancia.
6. **HTTPS**: instalar Caddy en la instancia con `Caddyfile` apuntando al dominio; renovación automática.
7. **Base de datos**: levantar contenedor MySQL con volumen EBS, ejecutar `schema.sql` inicial y las 12 migraciones existentes en orden.
8. **Variables de entorno/secretos**: dar de alta `JWT_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, credenciales DB en SSM Parameter Store; el deploy las inyecta al arrancar los contenedores.
9. **Despliegue backend**: `docker compose up -d backend` con `STORAGE_USE_IAM=true`, `sslMode=VERIFY_IDENTITY` real.
10. **Despliegue frontend**: `docker compose up -d frontend` con `NEXT_PUBLIC_API_URL=https://app.aelb.es/api`.
11. **Migraciones**: aplicar `database/migrations/*.sql` pendientes, incluyendo `20260813_files_to_s3.sql` y ejecutar una vez el perfil `storage-migration` para migrar documentos/PDFs legacy antes de dar el proyecto por desplegado.
12. **Backups**: cron `mysqldump` diario -> S3, y DLM para snapshot semanal del volumen EBS.
13. **Monitorización**: métricas básicas de CloudWatch (gratuitas, CPU/red/disco) + alarma simple de espacio en disco/CPU; sin logs centralizados de pago.
14. **CI/CD**: pipeline de GitHub Actions descrito en §10, con rol OIDC de GitHub hacia AWS (sin claves de larga duración).

Cambios que se dejarían preparados en el repositorio: un `docker-compose.prod.yml`, un `Caddyfile` básico, y un script `deploy/migrate.sh` para aplicar migraciones pendientes de forma repetible.